



Process Book

Visualizing Adversarial Examples Across Trainable Models



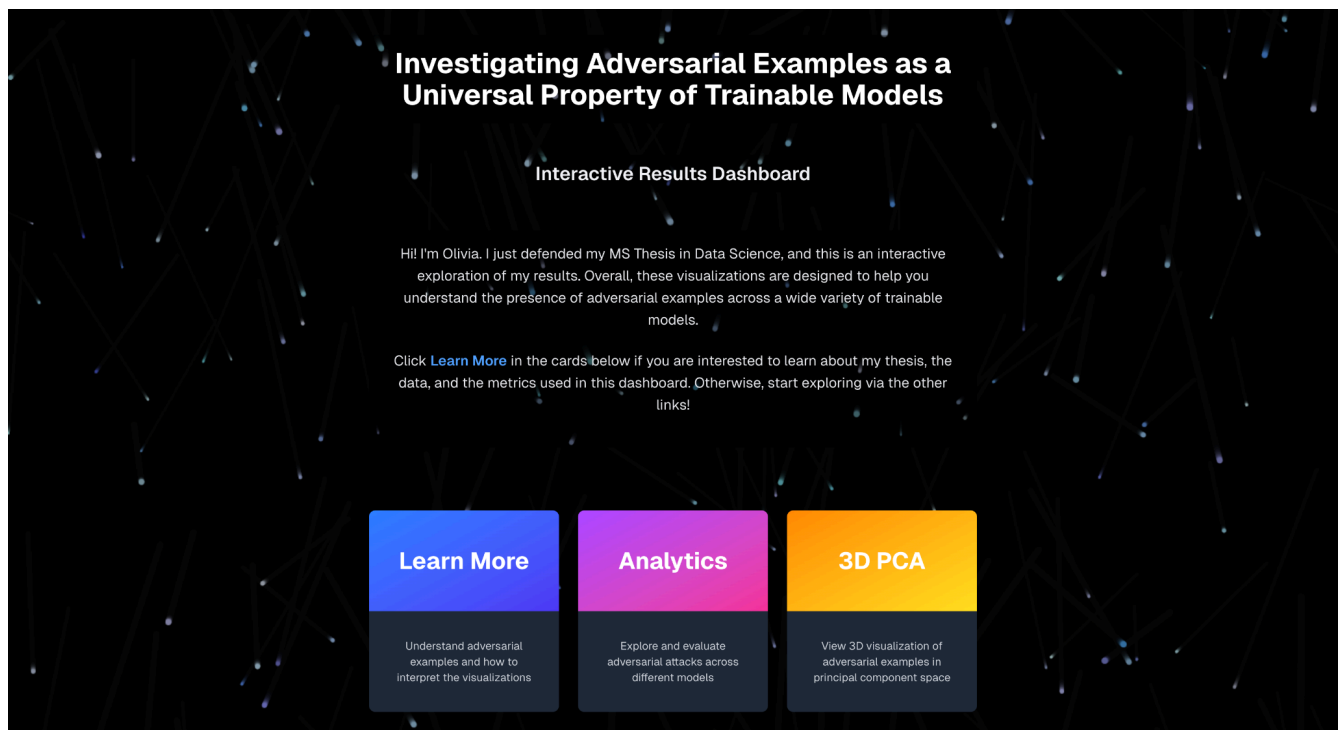
Olivia Cava

CS573 Final Project

Overview and Motivation

This visualization project aims to create an interactive dashboard to explore the presence of adversarial examples across different trainable models. Adversarial examples are inputs to machine learning models that are intentionally designed to cause the model to make a mistake (Goodfellow et al., 2015). My MS thesis investigated whether adversarial examples are a universal property across all trainable models.

The motivation for this visualization project is to make my research findings accessible and interactive. While the thesis contains detailed academic analysis, a data visualization dashboard allows others to explore the data themselves, filter results based on different parameters, and gain intuitive understanding of how adversarial examples affect different model architectures. Additionally, it's possible that even new things could be learned or new research directions can be discovered through having the ability to filter and interact with the data yourself. By creating clear visualizations of metrics like perturbation severity (using Frobenius norm) and attack success rates, I hoped to communicate the universal vulnerability of trainable models to these adversarial inputs, however to show that they can be mitigated through some intentional architectural choices.

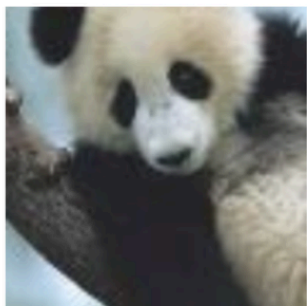
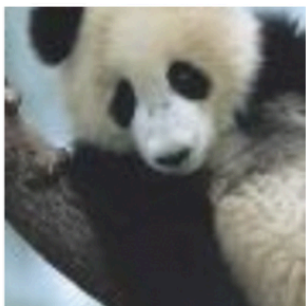


Related Work

Many works inspired this visualization project, both academic papers and certain interactive visualizations...

Academic Papers

My visualization approach was influenced by Goodfellow et al.'s "Explaining and Harnessing Adversarial Examples" (2014) paper, which provided the first accessible explanation of the adversarial phenomenon with clear visual examples. Their super famous and very simple "panda/gibbon" visualization shows the imperceptible perturbations applied (Goodfellow et al., 2014). This inspired my decision to include a perturbation explorer where you can compare the original image with its adversarial example counterpart. Additionally a 2016 paper by Tanay and Griffin inspired my 3D PCA visualization to show spatial relationships between original and adversarial examples. They discuss a lot about spatial relationships (proximity to decision boundaries) having great influence on robustness to adversarial attacks (Tanay & Griffin, 2016). Szegedy et al.'s "Intriguing Properties of Neural Networks" (2014) guided my decision to visualize robustness across different model architectures rather than just focusing on a single model.

	$+ .007 \times$		$=$	
x		$\text{sign}(\nabla_x J(\theta, x, y))$		$x + \epsilon \text{sign}(\nabla_x J(\theta, x, y))$
"panda"		"nematode"		"gibbon"
57.7% confidence		8.2% confidence		99.3 % confidence

^ Here is the famous "panda gibbon" viz from the Goodfellow et al. 2015 paper.

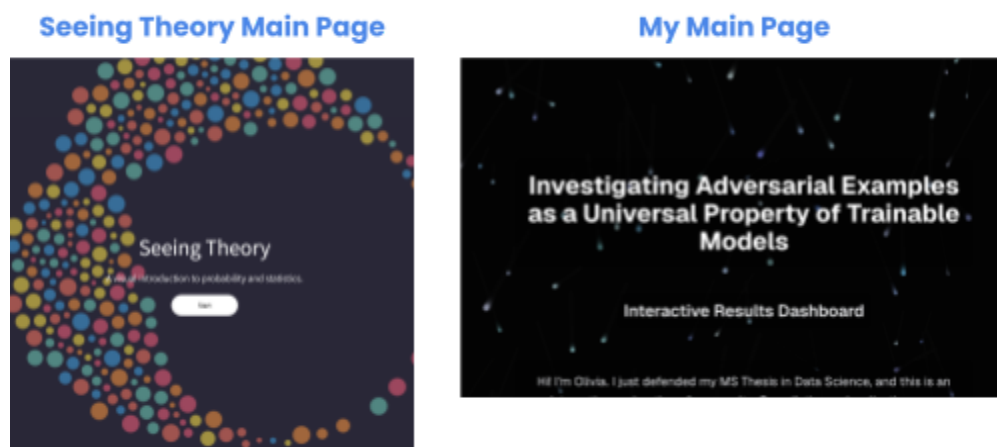
Interactive Visualizations

For interactive dashboard design, I was inspired by:

1. "Distill.pub" publications, especially their interactive machine learning visualizations that allow users to manipulate parameters and see results in real-time. This particularly inspired my Original vs Adversarial Example visualization where you can directly compare them next to one another.



2. The "Seeing Theory" statistics visualization website, which just overall influenced my approach to making abstract concepts visually intuitive, and also made me want to include a fun dot animation on the main page.



All Other Inspiration

Edward Tufte's visualization principles informed my approach to creating "small multiples" in the box plot/analytics page visualization, where I show the same metric

(Frobenius norm, % success, etc.) across different model types to facilitate intuitive comparison. The principles of effective data visualization covered in class lectures on color theory motivated my architecture color grouping scheme to make sure related models share consistent visual language.

Questions

Initial Questions

When starting this project, I wanted to answer:

1. How can I effectively visualize the severity of adversarial perturbation (Frobenius norm) across different model architectures?
2. How can I show success rates of adversarial attacks in an intuitive way?
3. How can I visualize the adversarial examples in a lower-dimensional space to show their relationships to original examples?
4. **Main Question:** How can I make the learning experience improved from my previous static visualizations in my MS thesis?

Evolution of Questions

As I worked with the data and created initial visualizations, my questions evolved to include:

1. How can I display model groups and their iterations in a hierarchical way that shows both individual model performance and group trends?
2. What is the most effective way to allow users to filter data based on attack parameters like KLD threshold?
3. How can I incorporate animated transitions to show the generation of adversarial examples over training iterations?
4. How can I create an interface that works for both domain experts and those new to adversarial example concepts?

New Questions

During implementation, I developed new questions:

1. How can I visualize the relationship between perturbation severity and attack success rate in a single view?
2. Can I create visualizations that show not just the statistical metrics but the actual visual changes in images through adversarial perturbation?
3. How can I most effectively pull in data that is large, and overcome limitations incurred from utilizing the free version of Supabase?

Data

Data Source

The data for this visualization comes directly from my MS thesis research, where I conducted experiments generating adversarial examples against various models. I created a dataset of 40,440 rows containing original images, their adversarial counterparts, and metrics for measuring attack effectiveness.

Data Collection Method

I collected this data by:

1. Training multiple model architectures with different configurations:
 - Multilayer Perceptron (MLP) baseline
 - Autoencoders with bottleneck sizes of 64, 128, 256, and 512
 - Autoencoder with no bottleneck
 - Hadamard product model
2. For each architecture, I trained multiple “iterations” (1-10) to analyze how vulnerability changes with training. For context, iterations here are distinct from training epochs and derive inspiration from dynamical systems theory, if you are interested in what I mean, feel free to check out my MS Thesis!
3. For each model, I ran 900 gradient based adversarial attacks on a set of 45 different models, collecting both the resulting adversarial images and metrics measuring the attack.

Data Cleanup and Processing

The initial data processing presented several challenges:

1. First, I tried reading in the data as is but realized I would need to paginate and process in batches due to the large size (40,440 rows with image vector data). I implemented a pagination system that fetches data in chunks of 1000 rows to avoid browser memory issues.

2. I noticed some inconsistencies in Frobenius norm calculations, requiring me to validate each value and filter out any invalid entries (NaN or undefined).
3. To organize the models logically, I created a model configuration system that groups models by their architecture type, assigns consistent colors, and maintains a consistent ordering for visualization.
4. For the 3D PCA visualization, I had to pre-compute PCA projections for each model's data and store them as separate JSON files, as calculating PCA transformations on the fly would be too computationally intensive for a web application. Future implementation could involve a python server with high compute power.

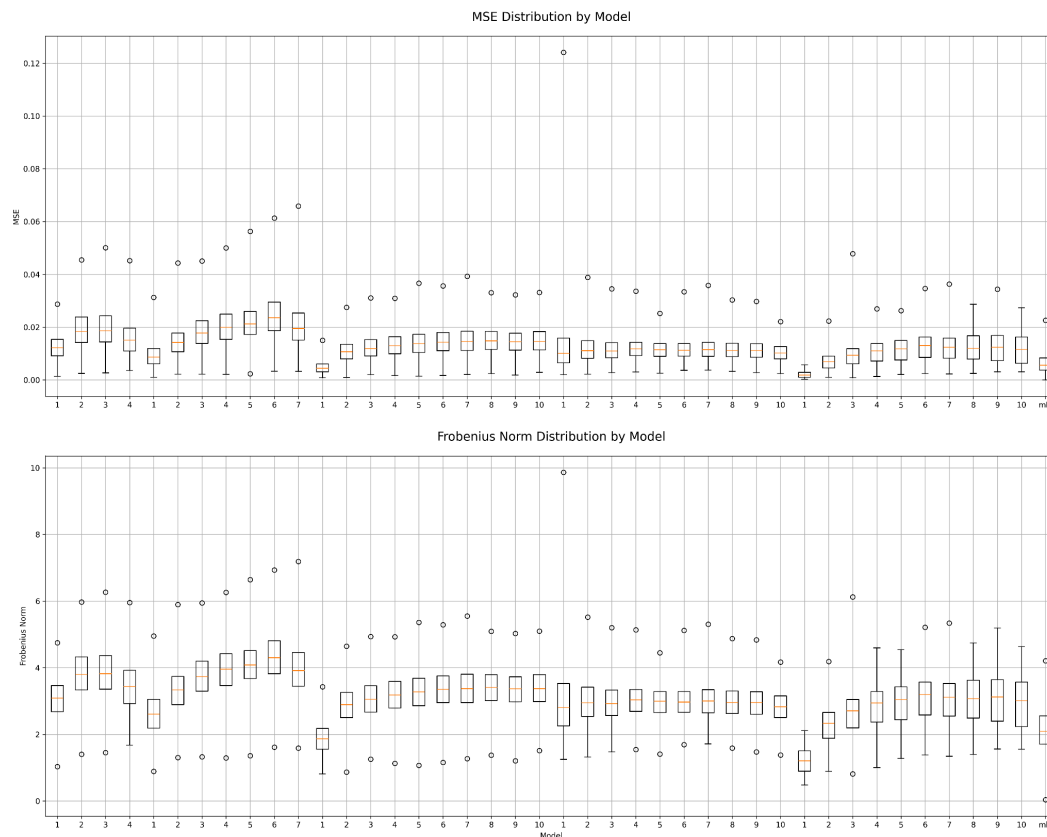
Column	Type	Description
id	BigInt	Unique identifier (Primary Key, Identity)
created_at	Timestamp	Creation timestamp (UTC)
case_idx	Integer	Adversarial case [0,899]
model_name	Text	Name of the model
image	Float[]	Original image array
label	Integer	True label
original_prediction	Float[]	Prediction on Original image
adversarial_image	Float[]	Adversarial image array
prediction	Float[]	Prediction on adversarial image
label_kld	Double precision	Final KL divergence for prediction and target DoC (encoded in case_idx)
mse	Double precision	Mean squared error of image and adversarial_image
frob	Double precision	Frobenius norm of image and adversarial_image

^ Here is my database schema for my research data hosted in Supabase..

Exploratory Data Analysis

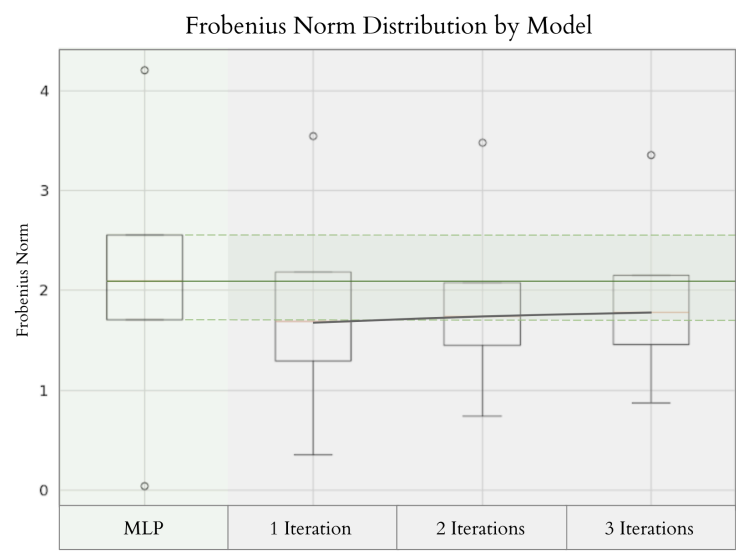
Initial Visualizations

When I first started exploring the dataset, I created several basic visualizations to understand the data distribution, and also had a bunch of visualizations left over from my MS Thesis that were static. I began with simple box plots plotting the Frobenius norm distribution for each model to understand the range and spread of perturbation magnitudes. Although I am embarrassed to admit, for my thesis I just took these bland visualizations and used Google Drawings to color over them and change formatting!

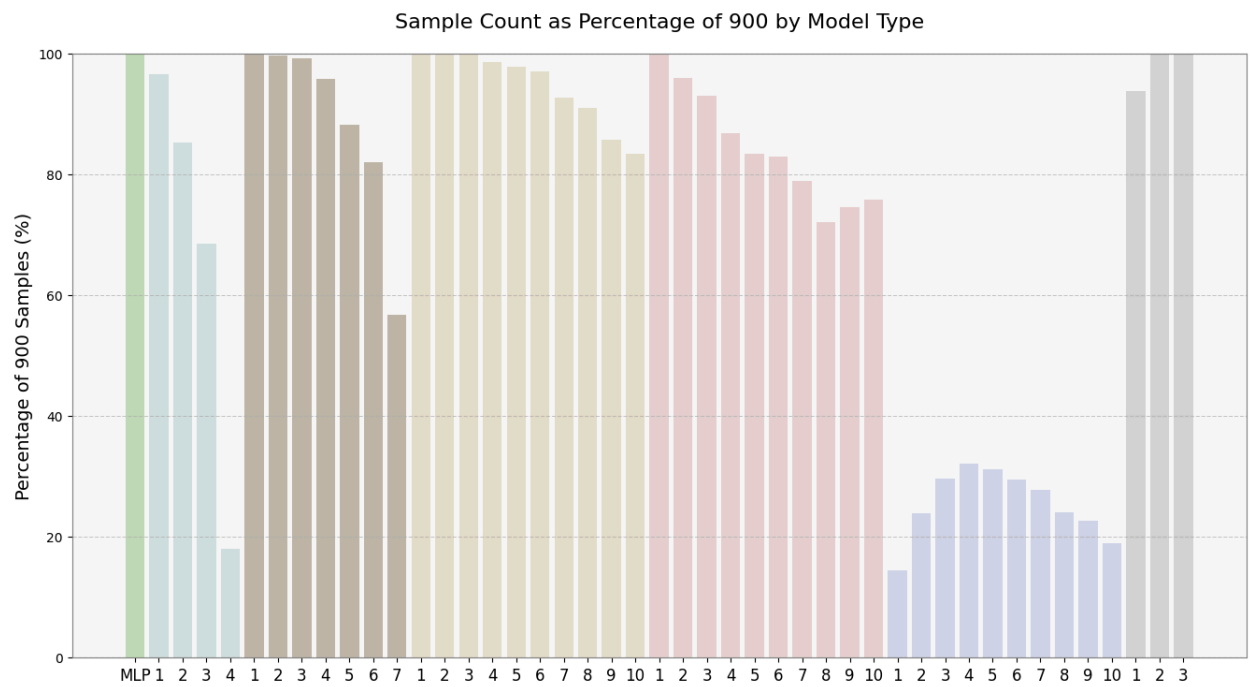


These plots revealed that different model architectures had distinctly different distributions of perturbation magnitudes, which suggested varying levels of robustness to adversarial attacks. I then colored over them with Google Drawings,

and used that colored version as inspiration when creating the interactive one. Below is an example of one I colored over using Google Drawings.



Additionally, I plotted the percent success in a static visualization and did the same color coding process as you can see below:



Insights Gained

From these exploratory visualizations, and through working on my thesis, I discovered that the MLP baseline model was more vulnerable to adversarial attacks than the autoencoder models, as it required much smaller perturbations for successful attacks. However, the inability to filter this data left me with more questions, hence the need for interactivity. Within autoencoder architectures, those with larger bottlenecks (512, 256) generally showed greater robustness than those with smaller bottlenecks (128, 64). Model robustness often changed non-linearly with training iterations, sometimes showing surprising patterns where early iterations were more robust than later ones, which you could see with the percent success visualizations. The Hadamard product model had different behavior from autoencoder architectures. This phase left me with questions like: *what do these look like in PCA? And, can we see these metrics (Frob and % success) at once in a single visualization?*

Influence on Design

These insights directly informed my visualization design decisions:

1. I decided to create a color-coded boxplot for Frobenius norm to clearly show the distribution differences between model architectures.
2. I implemented model grouping with color-coding to emphasize the architectural differences while showing individual model iterations.
3. I added interactive filtering based on KLD threshold to allow exploration of how different attack parameters affect the results.
4. I designed the 3D PCA visualization with an "ITERATE" button to show how adversarial examples evolve across training iterations.
5. I created a comparative visualization showing both perturbation severity and attack success rate in a single view to highlight their relationship.

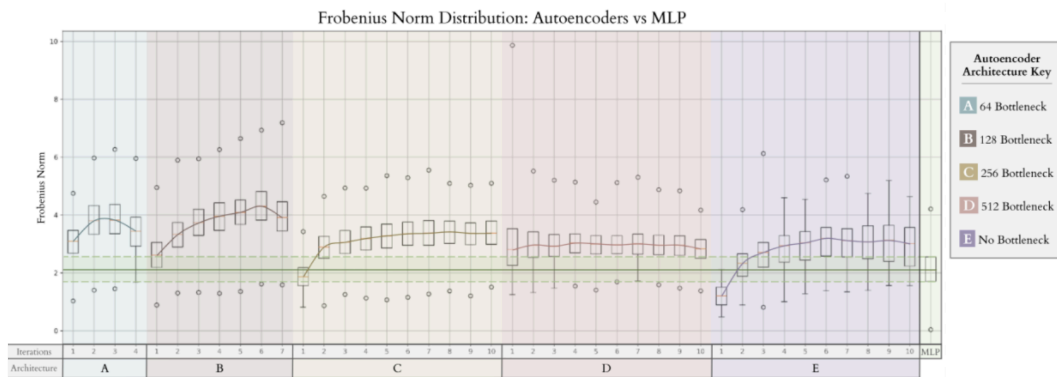
Design Evolution

First Mock-Ups

Filtering conditions:



FILTER!



Filtering conditions:



Digit



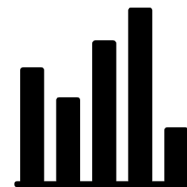
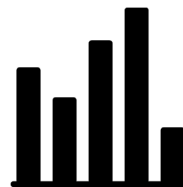
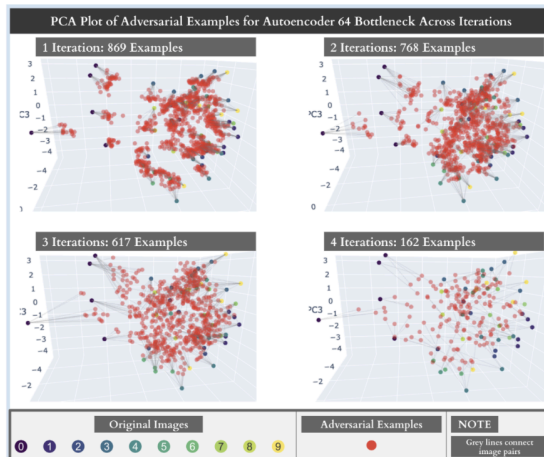
DoC

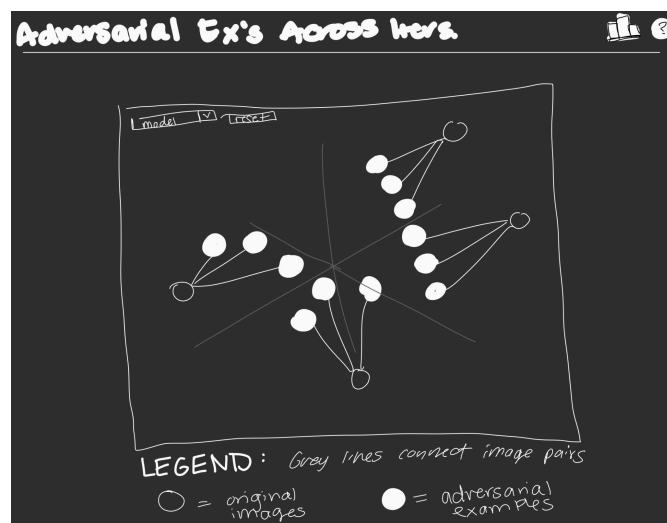
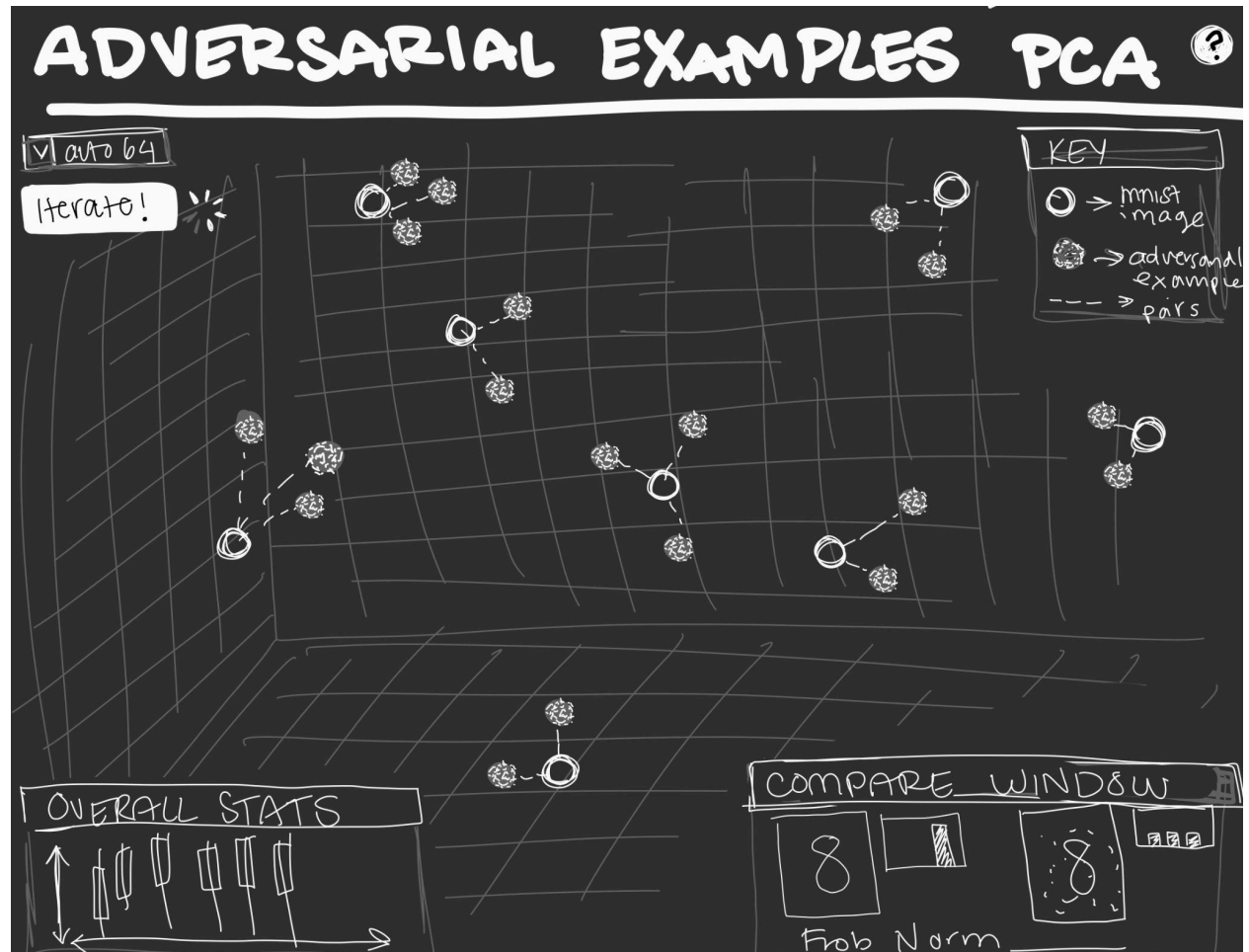


Includes True class

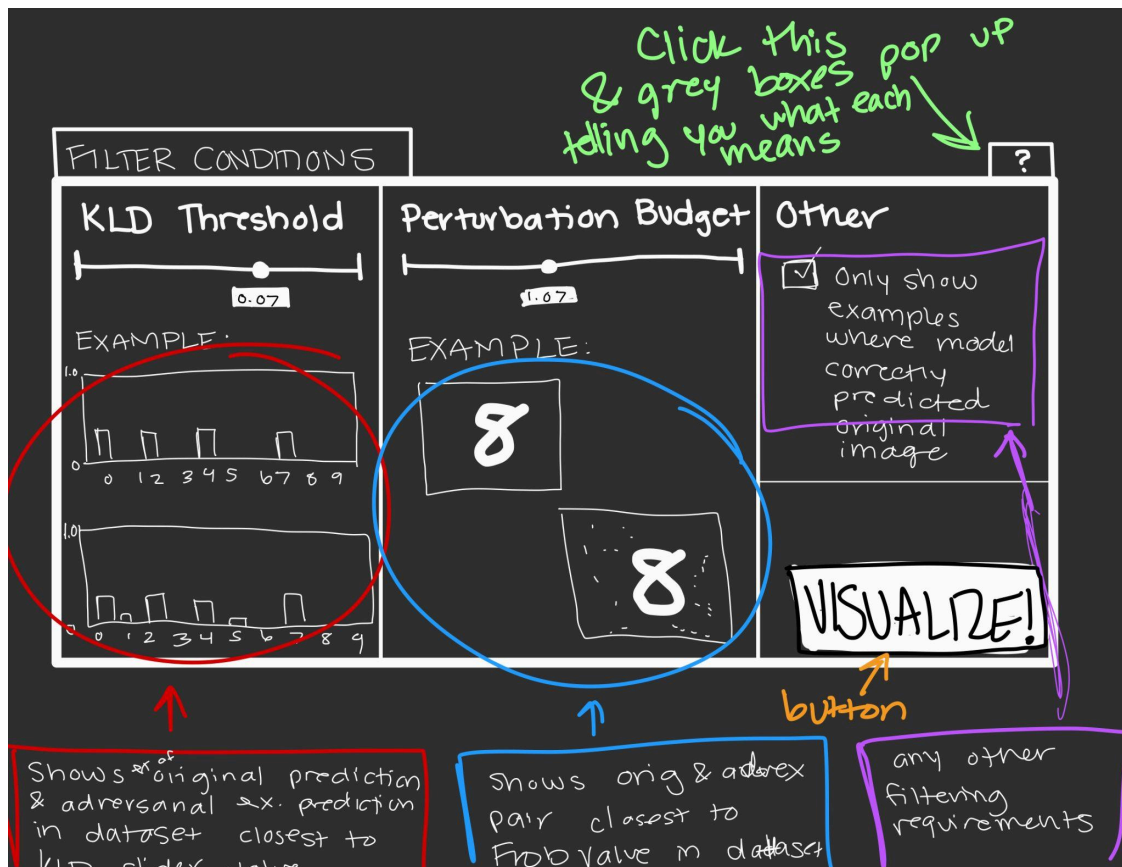


Which model?

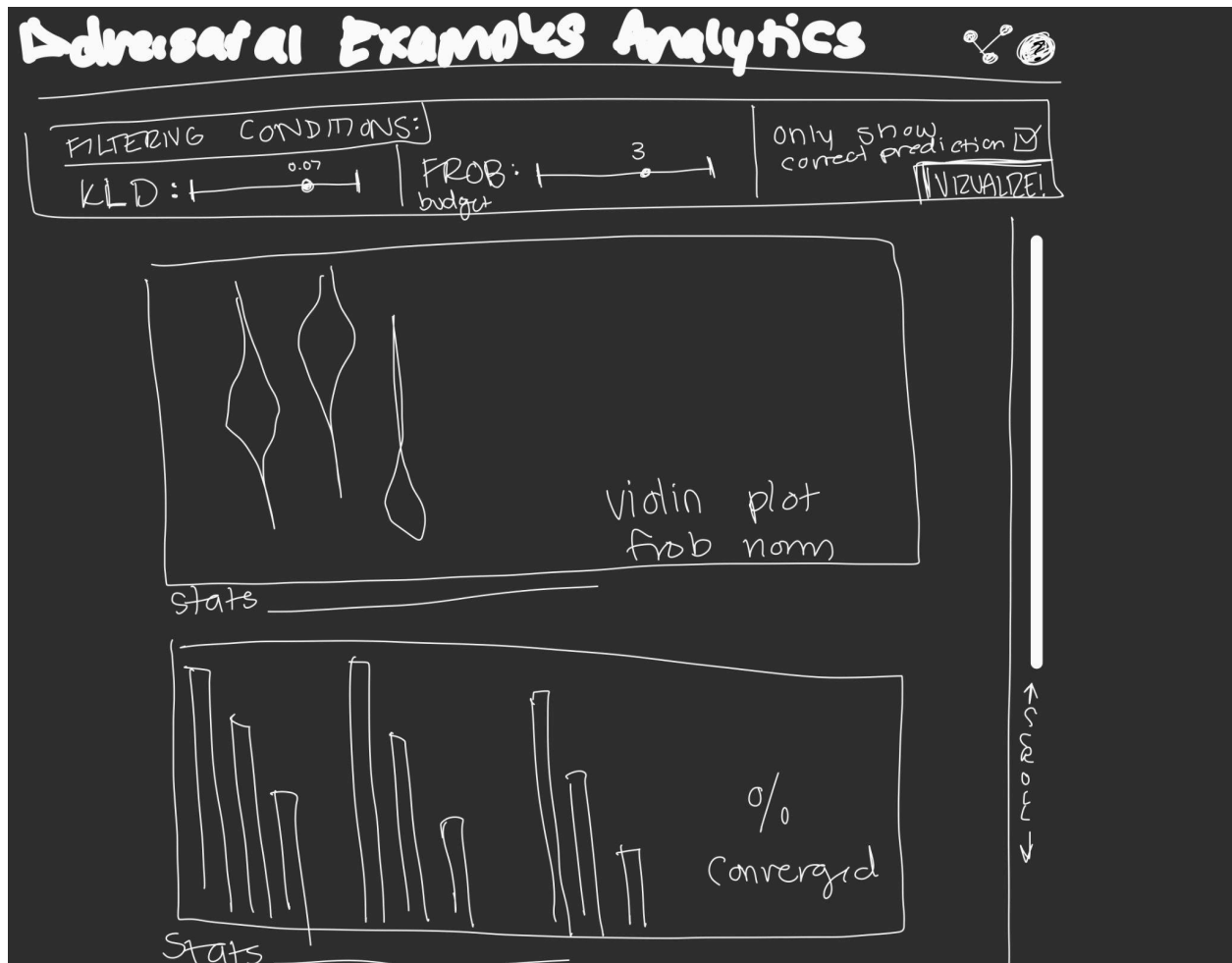




Explore Frobenius Norm Visualization Idea Sketches

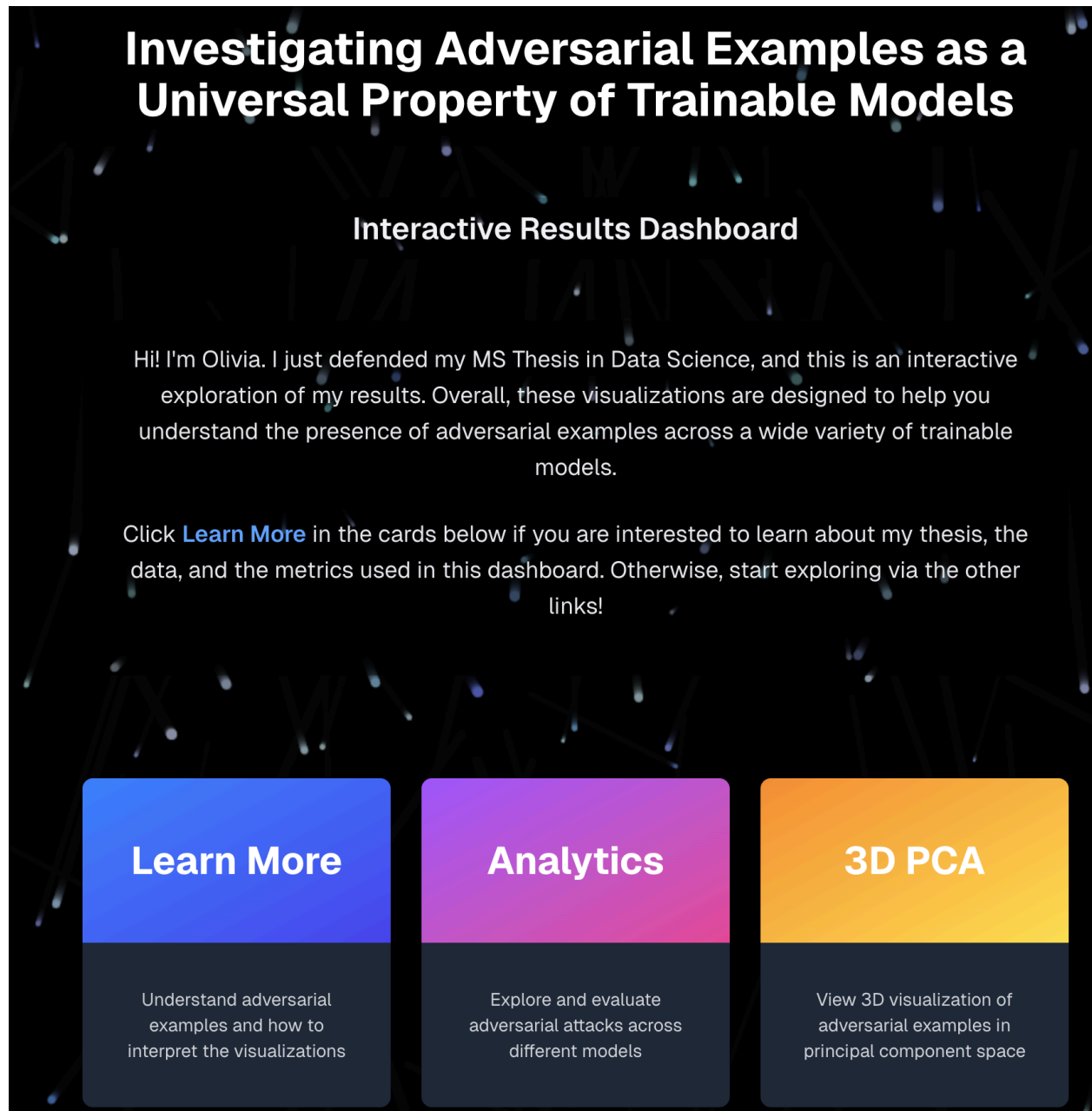


Analytics Page with One Main Filter Idea Sketch



Final Design Screenshots

Main Page

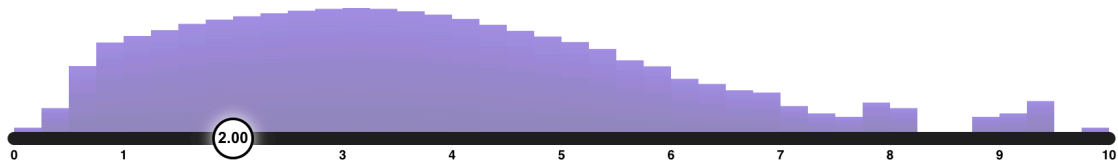


Perturbation Explorer

The Visual Impact of Perturbation Magnitude

Move the slider to compare the impact of various Frobenius (Frob) norm. This visualization pulls from the entire dataset of adversarial examples, and displays the original image and adversarial example pair with a Frob value closest to the one you selected. The histogram shows the distribution of Frob norms across the dataset (40,440 rows).

Distribution of Frobenius norm values across entire dataset



Original Image

Actual Frob: 1.90



Adversarial Example

Perturbation Severity Visualization

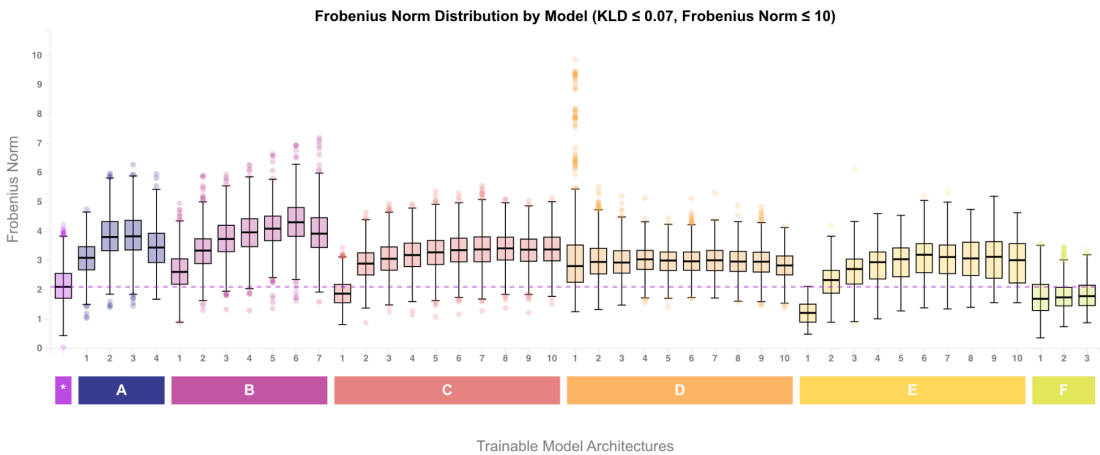
Filter Controls

KLD Threshold:0.07

Frobenius Norm:10

FILTER

Showing 29,956 examples across 45 models



Visualization Explanation:

- Each box represents the interquartile range (IQR) from 25th to 75th percentile
- The line inside the box shows the median value
- The whiskers extend to the minimum and maximum values (excluding outliers)
- Dots represent outliers (values more than 1.5 × IQR from the box edges)

NOTE: Models with **higher Frobenius norm** values are generally **more robust** against adversarial attacks, as they require larger perturbations to cause misclassification.

Percent Successful Attacks

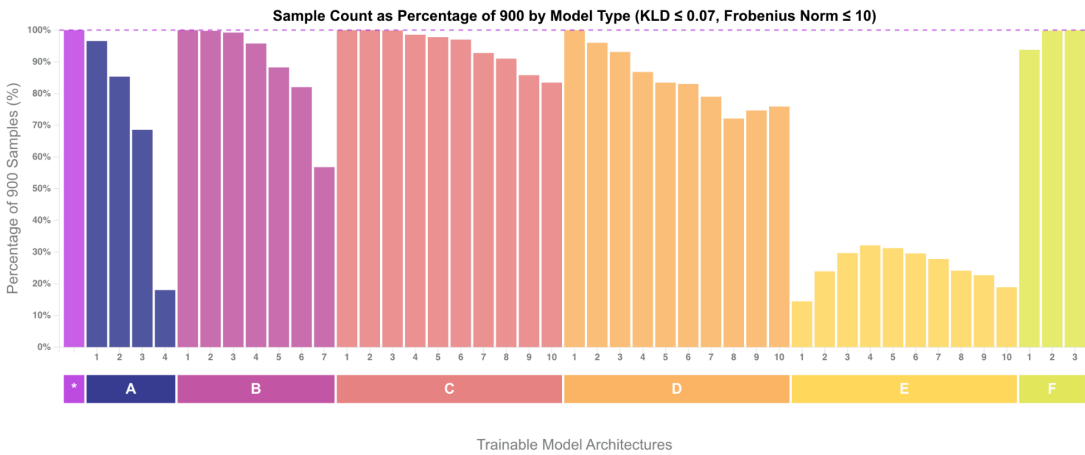
Filter Controls

KLD Threshold:0.07

Frobenius Norm:10

FILTER

Average success rate: 74.0% across 45 models

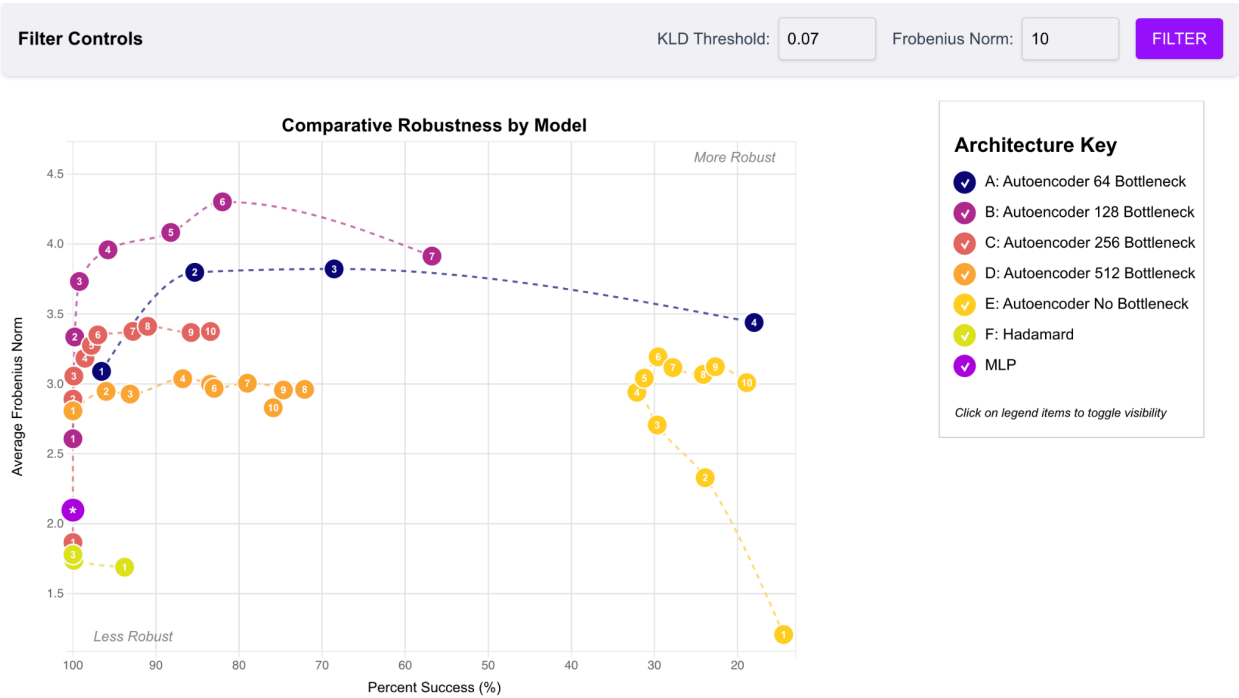


Visualization Explanation:

- Each bar represents the percentage of successful adversarial attacks out of a total of 900
- The purple dotted line shows the baseline MLP model percentage for comparison
- Hover over bars for exact counts and percentages

NOTE: Models with **lower percentages** were harder to attack and had fewer adversarial examples meeting the KLD and Frobenius norm thresholds, indicating better performance and robustness.

Combined Visualization

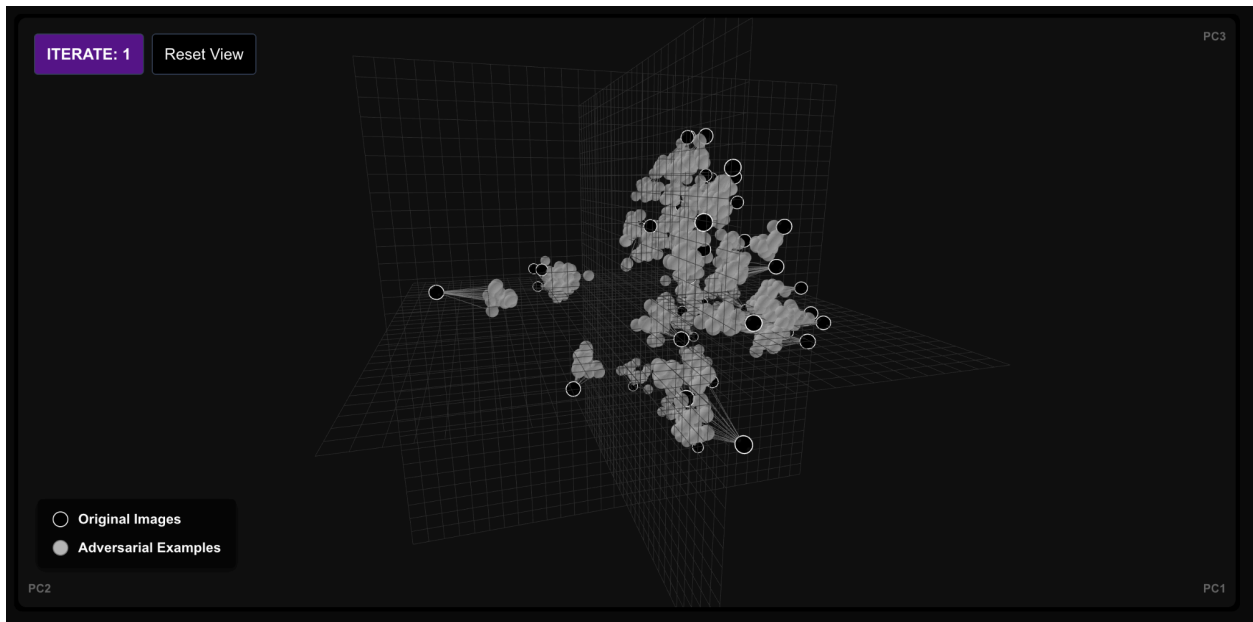


Visualization Explanation:

- The x-axis shows the percentage of successful adversarial attacks (100% @ the origin, lower values = more robust models)
- The y-axis displays the average Frobenius norm (higher values = more robust models)
- Points are connected by dashed lines to show progression across iterations
- Points heading towards the top right corner indicate a model becoming more robust across iterations
- Numbers inside points indicate the iteration number for that model architecture

NOTE: Models towards the upper-right corner display the **best robustness** to attacks with higher Frobenius norm and lower percentage of successful attacks.

PCA Visualization



Design Justification

My design decisions were based on several key principles. I used the same colors for models across all visualizations. This helps users connect information between different views. I created a color scheme that works for colorblind users. I used size and color to highlight important differences. These visual cues are processed quickly by viewers. The site has an overview page that shows the main concept first. Then users can explore more detailed visualizations. I added interactive controls that update visualizations in real-time. Users can change settings and see results immediately. Each visualization shows different aspects of the data. They all use the same naming and filtering systems. This creates a unified experience. Making everything connected while showing different data aspects was challenging.

Deviations from Proposal

The final implementation expanded beyond my original prospectus in several ways. My prospectus focused primarily on creating a 3D PCA visualization with basic filtering. The final project includes three additional visualizations I hadn't originally planned. I built a comprehensive Frobenius norm boxplot, a percent success visualization, and a combined comparative robustness view. I also created a more structured application with separate pages for each visualization type. The help section is more extensive than planned, with embedded video explanation and detailed documentation. I added more interactive elements throughout, including dynamic filtering, tooltips, and visualization controls. The original prospectus mentioned creating a simple "interactive sandbox," but the final implementation gives a much more polished learning experience. These additions better support my goal of making complex research findings accessible and interactive for both technical and non technical audiences, and inspiring potential future research.

Implementation

I built a Next.js web application with four main pages. The Homepage introduces the project with navigation cards. The Help Page provides background information on adversarial examples. The Dashboard Page contains three main visualizations. The 3D PCA page shows adversarial examples in principal component space. The Frobenius BoxPlot shows perturbation distributions across model types, and it groups models by architecture and uses color coding. It has interactive tooltips and filters. The Percent Success visualization shows attack success rates as a bar chart. It maintains consistent colors and includes tooltips. The Comparative Robustness View plots both metrics together. Models are positioned based on robustness. The 3D PCA visualization uses Three.js. It shows points in 3D space with interactive rotation. I faced several technical challenges during implementation. The large dataset required pagination to load properly. The 3D visualization needed optimization to handle thousands of points. Making filters responsive required careful state management. Browser compatibility and mobile responsiveness needed special attention.

Evaluation

My visualizations revealed several key findings. Different architectures have varying levels of robustness to attacks. The color coding makes this immediately clear. Model robustness changes nonlinearly with training iterations. Some models show increased vulnerability in middle iterations. The comparative view shows the inverse relationship between perturbation magnitude and success rate. The 3D visualization shows how adversarial examples move away from original images in feature space. My visualizations successfully answered my initial questions. They show perturbation severity, attack success rates, and the relationship between model complexity and robustness. The dashboard is effective in four main ways. It presents complex information clearly. It allows interactive exploration. It provides necessary context. It uses consistent design elements. Future improvements could include an image viewer for pixel level differences. More advanced filtering options could also help users explore specific patterns. Connected views with brushing and linking would

enhance interactivity. Performance optimization would create smoother interactions. Additional metrics would provide a more comprehensive view of adversarial robustness.

Conclusion

This project transforms complex research into an accessible dashboard. It focuses on clear visualization of key metrics with intuitive interactions. Both experts and not can explore the dashboard and understand model vulnerabilities. The visualizations communicate concepts in ways static papers cannot. Users can filter and interact with data directly. This creates not just a presentation but a fun tool for discovery!

Works Cited

Academic Papers:

Christian Szegedy et al. Intriguing properties of neural networks. arXiv:1312.6199 [cs]. Feb. 2014. DOI: 10.48550/arXiv.1312.6199. URL: <http://arxiv.org/abs/1312.6199> (visited on 02/07/2025).

Ian J. Goodfellow, Jonathon Shlens, and Christian Szegedy. Explaining and Harnessing Adversarial Examples. arXiv:1412.6572 [stat]. Mar. 2015. DOI: 10.48550/arXiv.1412.6572. URL: <http://arxiv.org/abs/1412.6572> (visited on 02/07/2025).

Thomas Tanay and Lewis Griffin. A Boundary Tilting Perspective on the Phenomenon of Adversarial Examples. arXiv:1608.07690 [cs]. Aug. 2016. DOI: 10.48550/arXiv.1608.07690. URL: <http://arxiv.org/abs/1608.07690> (visited on 04/02/2025).

Source Code & Helpful Documentation:

For the moving dots: <https://codepen.io/CELLFIX1/pen/qBwmEVJ>

d3.js boxplot: <https://d3-graph-gallery.com/boxplot.html>

d3.js barchart: <https://observablehq.com/@d3/bar-chart/2>

d3.js scatterplot: <https://d3-graph-gallery.com/scatter.html>

Misc Claude Prompts:

- How do I set up a Next.js project with TypeScript and Tailwind CSS for a data visualization dashboard?
- Create a React component for a homepage with a particle background effect and navigation cards to different sections.
- How can I implement a responsive navigation system between different visualization pages in a Next.js app?
- Write a React component that implements a boxplot visualization using D3.js to show data distributions.
- Help me create a function to fetch and process data from Supabase with proper pagination for large datasets.
- How do I handle error states and loading indicators for asynchronous data fetching in React?
- Write a D3.js implementation for a bar chart visualization showing percentage success rates.
- Create a D3 visualization that plots data points on two axes with size encoding for a third variable.
- How can I implement interactive tooltips for D3 visualizations that show detailed data on hover?
- Write a component for filtering data by KLD threshold and Frobenius norm with real-time updates.
- How do I implement color consistency across multiple visualizations for the same data categories?
- Create a basic Three.js implementation for visualizing data points in 3D PCA space.

- How can I implement camera controls (rotation, zoom, pan) in a Three.js visualization?
- Write code to add interactive elements like hover effects to 3D visualization points in Three.js.
- How do I create connecting lines between related data points in a Three.js 3D visualization?
- Create a function to calculate data bounds and normalize points for 3D visualization.
- Help me implement an "ITERATE" button that shows data progression across model iterations.
- How do I add axis labels and a legend to a Three.js 3D visualization?
- Write a component for a descriptive help page with embedded YouTube video and document links.
- How do I optimize rendering performance for visualizations with thousands of data points?
- Create a function to process and prepare PCA data for visualization from JSON files.
- How can I implement responsive design for complex visualizations that work on different screen sizes?
- Write a function to dynamically update visualization when filter parameters change.
- How do I implement a model configuration system to group and color-code different model architectures?
- Create a progress indicator for showing data loading status during pagination.
- Help me add a particle background effect for the homepage and help page.
- How do I deploy this Next.js visualization project to Vercel for sharing?